
IDG PMO

Museu das Amazônias 2026

Guia de Onboarding — Dashboard MAZ

Documento técnico de referência para novos desenvolvedores e responsáveis pelo projeto. Leia do início ao fim antes de fazer qualquer alteração.

Versão	Mai/2026 — v12
Projeto	pmo-creator/maz-dashboard
URL pública	pmo-creator.github.io/maz-dashboard
Responsável	IDG PMO

Índice

Índice.....	2
1. O que é este projeto.....	4
Arquitetura.....	4
2. Boas práticas de desenvolvimento.....	5
Ferramentas certas para cada tarefa.....	5
O que sempre fazer.....	5
O que nunca fazer.....	5
Boas práticas adicionais.....	5
3. Configuração inicial.....	7
Pré-requisitos.....	7
Clonar o repositório.....	7
Contexto para o Claude Code.....	7
Acesso ao GitHub.....	7
4. Alternativa visual — GitHub Desktop.....	8
Instalação e configuração.....	8
Clonar o repositório.....	8
Publicar alterações.....	8
5. Fluxo de trabalho — do teste ao ar.....	9
Passo 1 — Subir o ambiente de teste local.....	9
Passo 2 — Fazer as alterações.....	9
Passo 3 — Testar localmente.....	9
Passo 4 — Publicar.....	9
6. Testar no celular pela rede local.....	11
Encontrar o IP do computador.....	11
Acessar no celular.....	11
7. Reverter para uma versão anterior.....	12
Ver o histórico de commits.....	12
Opção A — Reverter um commit (recomendado).....	12
Opção B — Ver como o arquivo estava antes.....	12
Opção C — Voltar o repositório inteiro (cuidado).....	12
Pelo GitHub (interface visual).....	12
8. Referências técnicas.....	13
URLs.....	13
Google Sheets.....	13
API Key Google Sheets.....	13
Colunas das planilhas (índices 0-based).....	13

Regras de auto-status.....	14
Auto-refresh.....	14
9. Armadilhas técnicas conhecidas.....	14

1. O que é este projeto

Dashboard interativo do **Museu das Amazônias 2026 (MAZ ELD)** — acompanhamento de cronograma, status report e requisições de compras.

Arquitetura

O projeto inteiro consiste em dois arquivos HTML

- index.html — versão desktop
- mobile.html — versão mobile (3 abas: Gantt, Status Report, Requisições)

Ele funciona apenas no frontend, ou seja, toda a interface e a lógica de exibição rodam no navegador. Não há backend próprio, que seria a camada no servidor responsável por processar dados (bancos de dados), aplicar regras de negócio, autenticar usuários ou integrar sistemas.

Os dados são buscados direto do Google Sheets via API Key* no browser. O mesmo link público detecta o dispositivo e redireciona para a versão correta.

* API é uma interface que permite a comunicação entre sistemas. Ela serve para que uma aplicação solicite, envie ou consulte dados e funcionalidades de outra de forma padronizada.

Fluxo de dados

```
Google Sheets (Cronograma + REQS de Compras)
  | API Key (sem OAuth não precisa de login do usuário)
  | Browser do usuário
  | renderiza
  | Dashboard (index.html ou mobile.html)
```

Indicador de status (canto superior direito)

Indicador	Significado
Ao vivo · HH:MM	Tudo funcionando, dados frescos.
Cronograma OK · REQ erro	Cronograma OK, mas a busca de Requisicoes falhou.
Erro — dados locais	Fetch falhou, mostrando dados antigos em cache.

Sobre o indicador vermelho

- Se aparecer "Erro — dados locais", não é bug do dashboard. Verifique o compartilhamento da planilha e a validade da API Key.

Datas automáticas de marcos e grupos

As datas de início e fim de marcos e grupos são calculadas automaticamente pelo dashboard, sem depender das colunas de data da planilha para esses níveis:

- **Marcos:** início = data mínima das subtarefas; fim = data máxima das subtarefas. Marcos sem subtarefas mantêm as datas da planilha.
- **Grupo:** início = mínimo dos marcos; fim = máximo dos marcos.

2. Boas práticas de desenvolvimento

Ferramentas certas para cada tarefa

Tipo de tarefa	Ferramenta	Por que
Editar HTML/CSS/JS	Claude Code	Acessa e edita arquivos diretamente.
Git commit / push	Claude Code	Roda bash e git.
Debug de código em arquivos	Claude Code	Le o arquivo real, não uma cópia.
Perguntas sobre o projeto	Chat ou Cowork	Sem necessidade de ferramentas.
Explicações gerais de tecnologia	Chat	Puramente conversacional.
Brainstorm / planejamento	Chat ou Cowork	Sem necessidade de arquivos.
Criar documentos Word/PDF/PPT	Cowork	Skills especializados.
Análise de dados / planilhas	Cowork	Skills de data analysis.

O que sempre fazer

Fazer sempre

- Testar local antes de qualquer push.
- Hard refresh (Ctrl+Shift+R) ao testar — evita servir versão em cache.
- Verificar o indicador "Ao vivo" depois de atualizar.
- Um commit por alteração, com descrição clara do que foi feito.
- Testar no celular antes de publicar.
- Verificar os dois arquivos (index.html e mobile.html) quando a mudança afeta ambos.
- Descrever o commit em português, informando o que foi alterado e por quê.
- Fazer backup manual antes de alterar algo complexo: copiar o arquivo para Dashboard/backups/ com a data no nome.
- Validar JavaScript com node --check após qualquer alteração no código. Extrair o bloco `<script>` para um arquivo .js temporário e rodar: `node --check arquivo.js`
- Ao criar ou alterar o filtro de responsável, verificar os 3 estados em desktop e mobile: (a) todos marcados → cronograma completo, (b) alguns marcados → mostra só os responsáveis selecionados, (c) nenhum marcado → conteúdo some e aparece mensagem de aviso verde.

O que nunca fazer

Nunca fazer

- Editar direto no GitHub pelo browser — vai direto para produção sem teste.
- Confiar só no botão "Atualizar" do dashboard — ele re-executa o JS em cache, não baixa HTML novo.
- Publicar sem testar no celular.
- Push sem mensagem de commit descritiva.
- Rodar git push --force na branch main.
- Alterar as constantes de API Key ou IDs de planilha sem confirmar com o responsável.

Boas práticas adicionais

- Nunca altere as constantes de API Key ou IDs de planilha sem confirmar com o responsável do projeto.
- Qualquer mudança na estrutura das colunas das planilhas exige atualização do código de parse (`_parseWBS` e `_parseREQS`).
- Ao testar no celular, use sempre o link do GitHub Pages — não o localhost, que não redireciona corretamente para mobile.

3. Configuração inicial

Pré-requisitos

Instalar uma vez na máquina:

Ferramenta	Link
Git	https://git-scm.com
Python 3.x	https://python.org
Claude Code (recomendado)	https://claude.ai/code

Verificar instalações:

Abra o **PowerShell**: tecla Windows → digite "PowerShell" → pressione Enter. Na janela que abrir, cole cada linha abaixo e pressione Enter. Se aparecer um número de versão, está instalado corretamente.

```
git --version
python --version
```

API Key para desenvolvimento local

A chave de API de produção está restrita ao domínio `pmo-creator.github.io/*`. Localhost retorna 403 propositalmente — isso é comportamento esperado. **Não é necessário criar uma chave local.** O dashboard carrega com os dados estáticos embutidos no HTML; o indicador ficará ☐ (dados locais), mas a UI funciona normalmente para testar layout e funcionalidades. Para verificar dados ao vivo, publique no GitHub Pages e acesse pela URL pública.

Clonar o repositório

1. Abra o PowerShell: tecla Windows → digite "PowerShell" → Enter.
2. Navegue até a pasta onde vai salvar o projeto (ajuste "SEU_NOME" para o seu usuário):

```
cd "C:\\Users\\SEU_NOME\\OneDrive\\Documentos\\GitHub"
```

3. Cole o comando abaixo e pressione Enter. A pasta `maz-dashboard\\` será criada automaticamente:

```
git clone https://github.com/PMO-creator/maz-dashboard
cd maz-dashboard
```

Após o clone, copie os HTMLs para a pasta Ambiente de Teste (ver §4 e §5). A estrutura de pastas será:

```
GitHub\
  Ambiente de Teste_Dashboard\  <- editar e testar aqui
    index.html
    mobile.html
    SERVE_DASHBOARD.bat
    CLAUDE.md
    01. Manuais\                <- PDFs dos manuais

  maz-dashboard\                <- commit e push aqui
    index.html
    mobile.html
    DEV_GUIDE.html
    ONBOARDING.md               <- este guia
    CLAUDE.md
    .claude\doc_sync\           <- skill doc-sync
```

```
.claude\code_audit\          <- skill de auditoria
```

Contexto para o Claude Code

O repositório inclui um arquivo `CLAUDE.md` na raiz. Ele é lido automaticamente pelo Claude Code ao abrir o projeto e contém todo o contexto técnico necessário (arquitetura, CSS, SVG, armadilhas conhecidas). Você não precisa fazer nada — funciona automaticamente.

Acesso ao GitHub

O responsável anterior precisa te adicionar como colaborador:

```
github.com/PMO-creator/maz-dashboard > Settings > Collaborators > Add people
```

4. Alternativa visual — GitHub Desktop

Nota

- GitHub Desktop é recomendado para quem não tem familiaridade com linha de comando.

Instalação e configuração

- Baixar em desktop.github.com e instalar.
- Abrir o GitHub Desktop e fazer login com a conta GitHub do projeto.

Clonar o repositório

- File → Clone repository → URL → colar <https://github.com/PMO-creator/maz-dashboard>
- Escolher pasta local fora do OneDrive (ex: `C:\Dev\maz-dashboard`).

Publicar alterações

1. Abrir GitHub Desktop.
2. Verificar arquivos modificados no painel esquerdo.
3. Escrever mensagem no campo "Summary" (ex: "Corrigir nome da aba REQS").
4. Clicar "Commit to main".
5. Clicar "Push origin".

5. Fluxo de trabalho — do teste ao ar

Passo 1 — Subir o ambiente de teste local

O repositório já inclui o arquivo `SERVE_DASHBOARD.bat`. Basta executá-lo com duplo clique.

```
SERVER_DASHBOARD.bat  
→ Abre automaticamente http://localhost:8765/index.html
```

Ou manualmente no terminal:

```
cd maz-dashboard  
python -m http.server 8765
```

Depois abrir no browser: `http://localhost:8765/index.html`

Passo 2 — Fazer as alterações

Editar `index.html` e/ou `mobile.html` com Claude Code ou VS Code.

Atencao

- Alteracoes que afetam layout, KPIs, logica de dados ou parsing de colunas normalmente afetam os dois arquivos. Sempre verifique ambos.

Passo 3 — Testar localmente

Alvo	URL
Desktop	<code>http://localhost:8765/index.html</code>
Mobile (simulado)	<code>http://localhost:8765/mobile.html</code>
Celular real	<code>http://[SEU-IP]:8765/index.html</code> (ver secao 5)

- Hard refresh (Ctrl+Shift+R) a cada alteracao.
- Verificar indicador "Ao vivo".
- Testar as funcionalidades afetadas pela mudanca.

Passo 4 — Publicar

So publicar quando aprovado localmente

Via Claude Code (recomendado): peça em linguagem natural — “commita e sobe para produção com a mensagem: [descrição]”. O Claude executa os 3 comandos abaixo por você.

Manualmente: clique com o botão direito dentro da pasta `maz-dashboard\` no Explorer → “Abrir no Terminal” → cole cada comando abaixo e pressione Enter:

```
git add index.html mobile.html  
git commit -m "Descricao clara do que foi alterado"  
git push
```

Aguardar ~2 minutos, depois confirmar em:

```
https://pmo-creator.github.io/maz-dashboard/
```

Fazer hard refresh (Ctrl+Shift+R) no GitHub Pages para confirmar.

6. Testar no celular pela rede local

Celular e computador precisam estar na mesma rede Wi-Fi.

Encontrar o IP do computador

```
ipconfig
```

Procurar por "Endereco IPv4":

```
Adaptador de Rede Wi-Fi:  
Endereco IPv4 . . . : 192.168.1.105    <- esse e o IP
```

Acessar no celular

No browser do celular digitar:

```
http://192.168.1.105:8765/index.html
```

O redirect automatico vai direcionar para mobile.html.

IP pode mudar

- O IP local muda quando a rede muda. Sempre verifique com ipconfig antes de testar.

7. Reverter para uma versão anterior

Ver o histórico de commits

```
git log --oneline
```

Exemplo de saída:

```
126406f  Corrigir nome da aba REQS: Compras P -> Compras Prod
38249e4  Adicionar indicador de status ao vivo
e2247e7  Corrigir nome da aba do cronograma
9a47d9c  REQS: atualizar ID da planilha
```

Opção A — Reverter um commit (recomendado)

Cria um novo commit que desfaz as mudanças. O historico fica intacto.

```
git revert 38249e4
git push
```

Opção B — Ver como o arquivo estava antes

```
git show 38249e4:Dashboard/index.html > versao_antiga.html
```

Abrir versao_antiga.html para comparar ou copiar trechos.

Opção C — Voltar o repositório inteiro (cuidado)

Atencao — operacao destrutiva

- Apaga tudo que foi feito depois desse commit. So usar em ultimo caso. Avisar o responsavel antes.

```
git reset --hard 38249e4
git push --force
```

Pelo GitHub (interface visual)

1. Acessar github.com/PMO-creator/maz-dashboard
2. Clicar na aba "Commits".
3. Encontrar o commit desejado > clicar em "<>" (Browse files).
4. Baixar o arquivo da versao antiga manualmente.

8. Referências técnicas

URLs

Recurso	URL
Dashboard publico	https://pmo-creator.github.io/maz-dashboard/
Repositorio GitHub	https://github.com/PMO-creator/maz-dashboard
Teste local desktop	http://localhost:8765/index.html
Teste local mobile	http://localhost:8765/mobile.html

Google Sheets

Planilha	ID	Aba
Cronograma	17nttJ_ShqWztvDWH3l59iNqboLqkviZs3_PM5J3ihdA	master data
Requisicoes	1azrdS4OGO-CWD1ods69i8iZJcwq4oylSdT2n_tu1uJM	Planilha de Status de Compras Prod

Permissao obrigatoria

- Ambas as planilhas precisam estar com "Qualquer pessoa com o link pode ver" ativado. Sem isso o dashboard retorna erro 403/400.

API Key Google Sheets

Campo	Valor
Chave	[solicitar ao responsável]
Projeto GCP	maz-dashboard-495414
Restricao	pmo-creator.github.io/*
Gerenciar em	console.cloud.google.com > APIs e servicos > Credenciais

Colunas das planilhas (índices 0-based)

Cronograma (_parseWBS)

Coluna	Indice	Campo
B	1	Eixo
C	2	Grupo
D	3	Marco
E	4	Tarefa
G	6	Responsavel
H	7	Status
I	8	Data inicio

Coluna	Índice	Campo
J	9	Data fim

Requisições (_parseREQS)

Coluna	Índice	Campo
A	0	N. Requisicao
B	1	Comprador
D	3	Prioridade
E	4	Descricao
F	5	Status
G	6	Fornecedor
M	12	Data prevista

Regras de auto-status

Condicao	Status resultante
Subtask sem status + sem data	Definir datas
A iniciar + sem data de início	Definir datas
Grupo/eixo sem status definido	Definir datas
Qualquer status + data fim já passou	Atrasado
Qualquer status + data fim nos próximos 7 dias	Risco de atraso
Feito, Cancelado, Cancelado/Congelado	Nunca muda

Rollup: marco = pior status das tarefas / grupo = pior status dos marcos / eixo = pior status dos grupos. **Ranking:** Atrasado(0) > Risco de atraso(1) > Em andamento(2) > Definir datas(3) > A iniciar(4) > Feito(5) > Cancelado/Congelado(6)

Auto-refresh

Intervalo: 15 minutos — constante `AUTO_REFRESH_MS` em ambos os HTMLs.

Filtros do Dashboard

Os filtros globais (Status, Eixo, Data e Responsável) ficam na barra superior do desktop (`index.html`) e como chips/dropdowns no mobile (`mobile.html`). Todos alimentam a função `applyFilter()` que re-renderiza a árvore EAP e o Gantt.

Filtro de Responsável — Desktop (`index.html`)

Localização no HTML: `<div class="ms-resp-wrap">` na barra de filtros, entre o filtro de Eixos e o botão “Limpar Filtros”.

Função JS	Responsabilidade
<code>buildRespFilter()</code>	Lê os nomes únicos de responsável dos dados e monta o dropdown multi-select.

<code>grupoHasResp(grupo, resp)</code>	Retorna true se o grupo ou qualquer uma de suas tarefas pertence ao responsável.
<code>getFilters()</code>	Retorna o objeto de filtros ativos, incluindo o campo resp (array de nomes selecionados).
<code>applyFilter()</code>	Detecta “nenhum marcado” via contagem DOM (não via f.resp), exibe #gantt-resp-msg e oculta linhas sem correspondência.

Lógica de filtragem (hierarquia):

- Marco com o responsável → mostra o marco com todas as suas tarefas.
- Marco sem o responsável, mas com alguma tarefa do responsável → mostra o marco só com as tarefas do responsável.
- Grupo sem nenhum marco/tarefa do responsável → desaparece.
- Eixo sem nenhum grupo com o responsável → desaparece.
- No Gantt: linhas sem o responsável são ocultadas (não esmaecidas).

Filtro de Responsável — Mobile (mobile.html)

Localização: chips horizontais com classe `.resp-chip` exibidos entre as abas e o conteúdo, nas abas Gantt e Status Report.

Elemento / Função JS	Responsabilidade
<code>activeRespFilterM</code>	Array de responsáveis ativos no mobile.
<code>buildRespFilterMobile()</code>	Gera os chips de responsável para o mobile.
<code>grupoHasRespM(grupo, resp)</code>	Versão mobile de grupoHasResp — mesma lógica, adaptada para o contexto mobile.
<code>#gantt-resp-msg</code>	Mensagem “SELECIONE AO MENOS UM RESPONSÁVEL” exibida na aba Gantt quando nenhum chip está marcado.
<code>#sr-resp-msg</code>	Mensagem “SELECIONE AO MENOS UM RESPONSÁVEL” exibida na aba Status Report quando nenhum chip está marcado.

Filtro de Fornecedor (aba Requisições)

A aba Requisições possui filtros específicos adicionados no commit b4a0b53:

- `buildFornecedorFilter()` — dropdown multi-select de fornecedor, gerado a partir da coluna G (índice 6) do sheet REQS. Itens sem fornecedor agrupados como “Sem Fornecedor”.
- `clearAllFilters()` — botão verde que reseta simultaneamente status, fornecedor, comprador, prioridade e campo de busca.

9. Armadilhas técnicas conhecidas

Armadilha	Como evitar
Dashboard branco sem erro no console	Verificar: (a) null bytes no HTML, (b) palavra "function" ausente em declaração JS, (c) JS truncado sem <code></script></code> , (d) template literals aninhados
Template literals aninhados	Nunca usar crase dentro de <code>\${}</code> dentro de outro crase — node --check passa mas browser quebra
JS truncado	Verificar se <code></script></code> existe no final do arquivo antes de editar

Prioridade REQS	Usar APENAS coluna D (índice 3) — coluna B gera falsos positivos. Commit 7662590.
node --check no Node v22	Não aceita .html — extrair bloco <script> para arquivo .js temporário
Mapeamento de colunas REQS desatualizado	Os índices de _parseREQS foram corrigidos (commit 7662590). Índices corretos: r[0]=N.Req, r[1]=Comprador, r[3]=Prioridade, r[4]=Descrição, r[5]=Status, r[6]=Fornecedor, r[12]=Data prevista. Se os KPIs de Requisições aparecerem zerados, verificar primeiro se o mapeamento de colunas está correto.

10. Skills disponíveis no repositório

O repositório inclui duas skills em .claude/ prontas para uso com Claude Code.

code_audit — Auditor de código

Analisa o `git diff` e reporta problemas antes de subir para produção — segurança, arquitetura, qualidade de código, fluxo git e dependências. É sugerida automaticamente pelo Claude Code antes de commit/push e ao adicionar dependências. Sempre pergunta antes de rodar.

Como acionar manualmente (escrever em linguagem natural no Claude Code):

O que digitar	O que faz
"audita o que mudou"	Analisa só o git diff atual — leve
"resumo do projeto"	Lê o CLAUDE.md e resume o estado — leve
"auditoria completa"	Lê todos os arquivos — pesado, usar com moderação

doc-sync — Sincronização de documentação

O projeto usa uma skill chamada **doc-sync** instalada no Claude/Cowork para detectar automaticamente mudanças no código do dashboard e atualizar os manuais e guias afetados. Roda todo dia às 08:00 enquanto o Cowork estiver aberto.

Instalação (primeira vez)

1	Localize o arquivo doc-sync.skill na pasta do projeto (IDG - Relatórios de Análise) e clique em “Save skill” no Claude Cowork.
2	No Cowork, abra uma nova sessão e digite: Configurar doc-sync para rodar todo dia às 08:00 . O Claude configura o scheduled task automaticamente.
3	Clique em “ Run now ” na aba Scheduled do sidebar para pré-aprovar as permissões. Isso evita que execuções futuras travem pedindo autorização.

Como invocar manualmente

Após qualquer **git push** do dashboard, abra uma sessão no Cowork e escreva **doc-sync**. A skill identifica automaticamente o que mudou, explica em linguagem simples e pede sua aprovação antes de alterar qualquer documento.

Arquivos da skill no repositório

A skill e seu contexto vivem no próprio repo GitHub, garantindo portabilidade para qualquer dev futuro:

Arquivo	Descrição
.claude\doc_sync\SKILL.md	Instruções completas da skill (o que ela faz e como)
.claude\doc_sync\context.md	Conhecimento completo do projeto (arquitetura, docs, regras de relevância)
.claude\doc_sync_snapshot_index.html	Última versão do dashboard documentada (linha de base para o próximo diff)
CLAUDE.md	Contexto rápido do projeto para qualquer Claude ao abrir o repo

Guia atualizado em 25/Mai/2026 — v12 — Dashboard MAZ 2026 · IDG PMO